
Ch 06 함수

Sec 04 자료교환

Sec 05 오버로딩

함수의 기본 사용법(1)

- 함수를 사용하려면 함수 선언, 함수 호출, 함수 정의 라는 엔티티가 필요
- 함수 정의(function definition)는 함수구현을 의미

```
리턴_자료형 함수_이름(매개변수_리스트)
{
    본문
}
```

그림 6-2 함수 정의 구문

argument와 parameter

parameter must be var !!!!!

```
void fun(int x) // 함수 선언
int main()
{
    ... argument : int literal or int var
    fun(5);      // 함수호출, 함수 호출에서 5가 argument
    ...
}
void fun(int x) // 함수 정의에서 x가 parameter
{
    ...
}
```

- parameter는 함수 정의에 있는 변수 선언을 뜻함
- argumen는 함수를 호출할 때 매개변수를 초기화하는 값, 변수, 리터럴 모두 가능

자료 교환

- 커뮤니케이션은 [그림 6-14]처럼 자료가 양방향으로 이루어짐

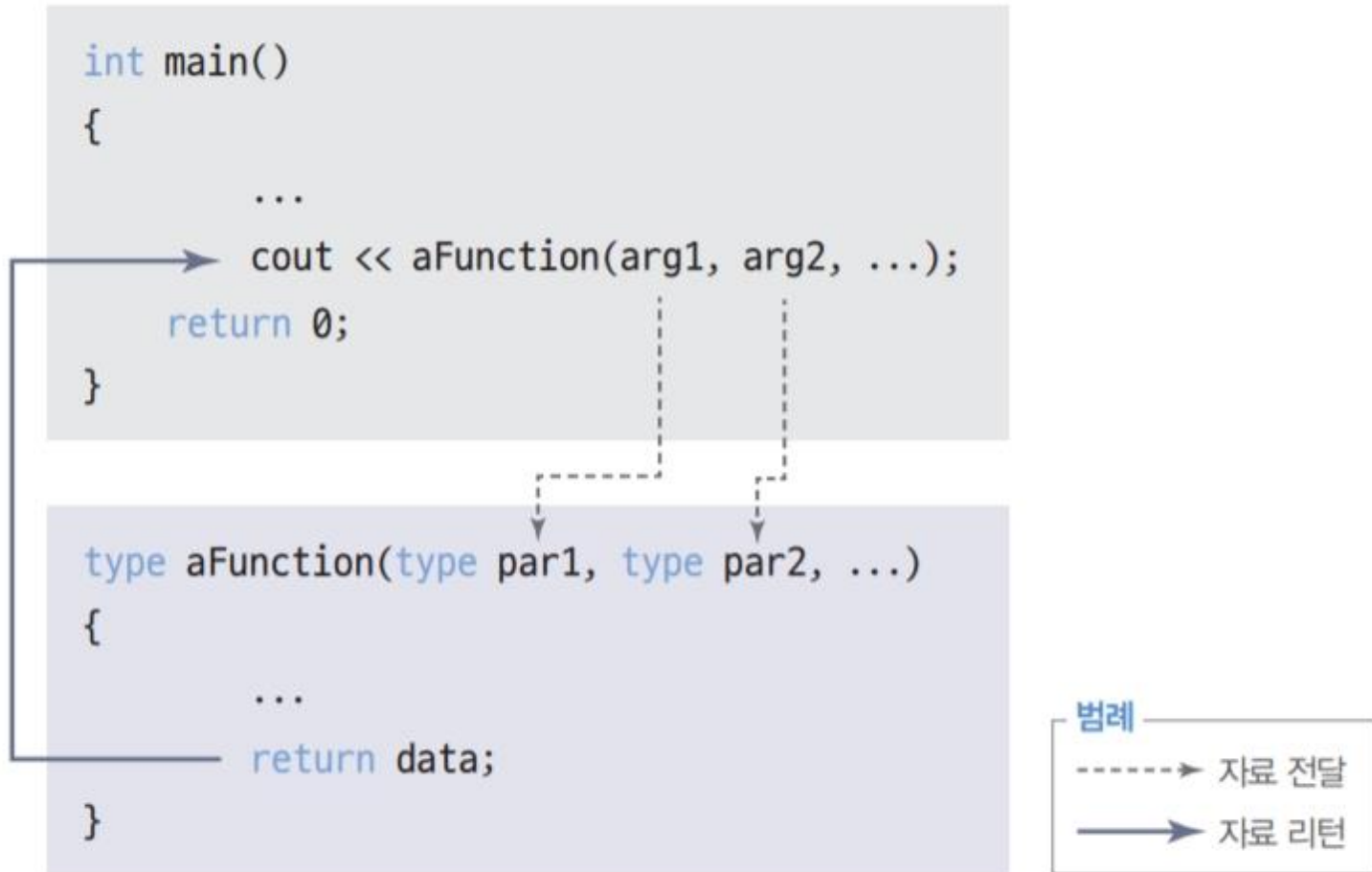


그림 6-14 함수 사이의 자료 교환

자료 전달(1)

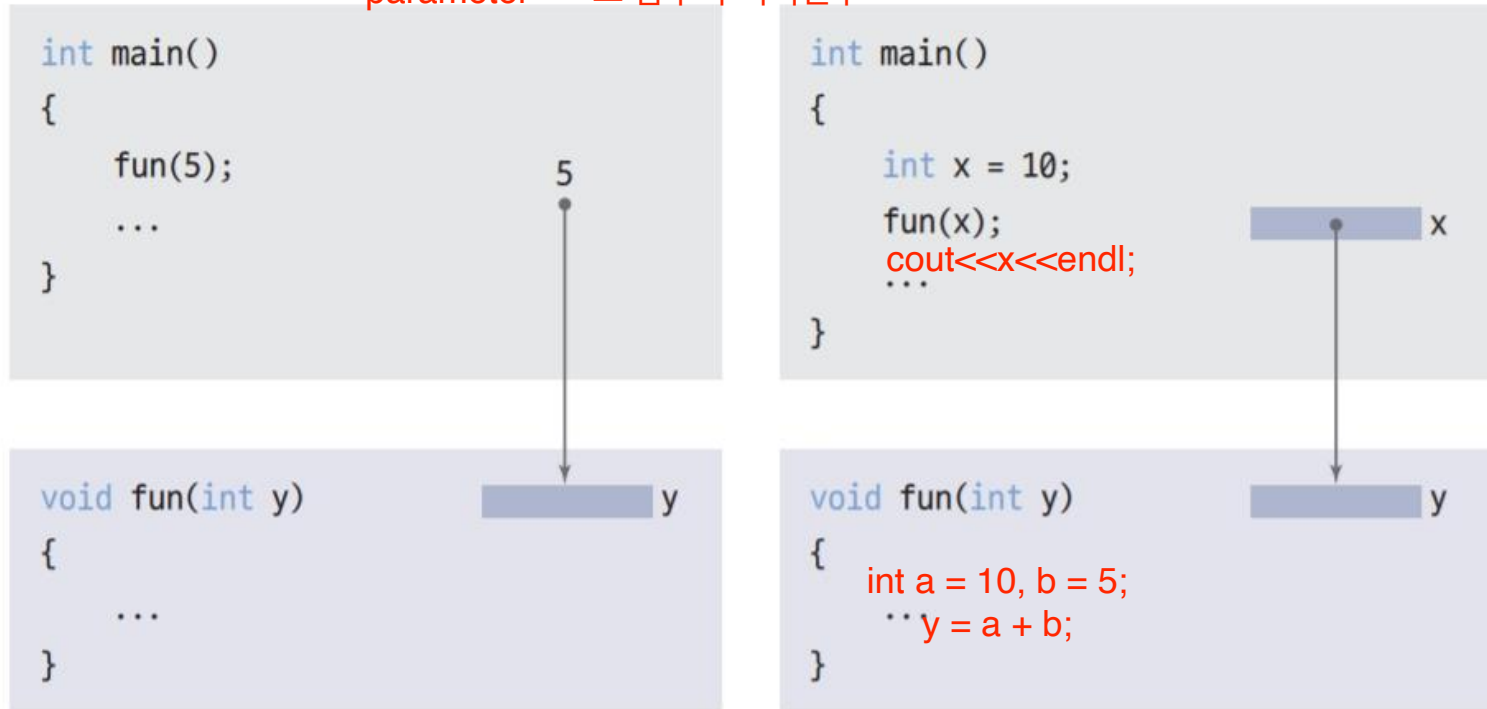
- 호출되는 함수에 매개변수가 존재한다면, 함수 호출 때의 값(argument)이 호출되는 함수의 매개변수(parameter)로 전달
- 자료 전달은 값으로 전달(**pass-by-value**), 참조로 전달(**pass-by-reference**), 포인터로 전달(**pass-by-pointer**)의 3가지 메커니즘으로 구분

자료 전달(2)

값으로 전달(pass-by-value) memory of stack 그리기 "read Only"

- 값으로 전달(pass-by-value) 메커니즘에서는 인수(argument)의 값이 복사되어서 매개변수(parameter)에 할당

parameter == 그 함수의 지역변수



(a) 리터럴 값 전달하기

(b) 변수의 값 전달하기

그림 6-15 값으로 전달(pass-by-value)의 개념

자료 전달(3)

- Pass-by-value는 함수에서 argument의 값을 변경할 수 없고, 단지 parameter가 값을 전달 받음
- 전달 메커니즘에서 호출되는 함수는 인수의 값을 읽기만 한다
- 이를 '읽기 전용 접근(read-only access)'이라고 표현

값으로 전달 확인하기(1)

프로그램 6-13 값으로 전달 확인하기

```
1  /*****
2  * This program shows how a change in the parameter cannot      *
3  * affect the corresponding argument in pass-by-value.          *
4  *****/
5  #include <iostream>
6  using namespace std;
7
8  // Function declaration
9  void fun (int y);
10
11 int main ()
12 {
13     // Declaration and initialization of an argument
14     int x = 10;
15     // Calling function fun and passing x as an argument
16     fun (x);
17     // Printing the value of x to see no change
18     cout << "Value of x in main: " << x << endl;
19     return 0;
20 }
```

값으로 전달 확인하기(2)

프로그램 6-13 값으로 전달 확인하기

```
21  /*****
22  * Fun is a function that receives the value of x by value and *
23  * stores it in parameter y. The locations x and y are two      *
24  * independent memory locations. The value of y is incremented *
25  * in fun, but the value of x in main remain unchanged          *
26  *****/
27  void fun (int y)
28  {
29      y++;
30      cout << "Value of y in fun: " << y << endl;
31      return;
32  }
```

Run

Value of y in fun: 11 // y in the called function is incremented.

Value of x in main: 10 // x in the main is not affected.

참조로 전달

Pass - by - Reference

```
int main()
{
    int x = 10;
    fun(x);
    ...
}
```

x

```
void fun(int& y)
{
    ...
}
```

y

참조변수를 통해서 main 함수 외부에서 var 을 변경 가능하다

참조변수 :

“새로운 메모리할당 L L”

“선언과 동시에 초기화 되어야 한다 !!!”

❗ fun 함수에서는 변수 y를 위한
메모리 공간이 추가로 할당되지 않음

매개변수로 전달된 변수 x의
메모리 공간을 사용

그림 6-16 참조로 전달(pass-by-reference)의 개념

참조로 전달(pass-by-reference) 확인하기(1)

프로그램 6-14 참조로 전달(pass-by-reference) 확인하기

```
1  /*****
2   * This program shows how a change in the parameter can change the *
3   * corresponding argument when data is passed by reference.      *
4   *****/
5  #include <iostream>
6  using namespace std;
7
8  // Function declaration;
9  void fun (int& y); // The ampersand tell us that y is an alliance
10
11 int main ()
12 {
13     // Declaration and initialization of an argument
14     int x = 10;
15     // Calling function fun and passing x as an argument
16     fun (x);
17     // Printing the value of x to see change
18     cout << "Value of x in main: " << x << endl;
19     return 0;
20 }
```

참조로 전달(pass-by-reference) 확인하기(2)

프로그램 6-14 참조로 전달(pass-by-reference) 확인하기

```
21  /*****
22  * Fun is a function that receives the value of y by reference,      *
23  * which means the parameter y is an alias for the argument x in    *
24  * the function call. This function increments its parameters which *
25  * results in incrementing the argument in main.                    *
26  *****/
27  void fun (int& y)
28  {
29      y++;
30      cout << "Value of y in fun: " << y << endl;
31      return;
32  }
```

Run

Value of y in fun: 11

Value of x in main: 11 // Change of y in fun, has changed x in main

추가적인 변수(temp)를 사용해서 두 변수 스왑

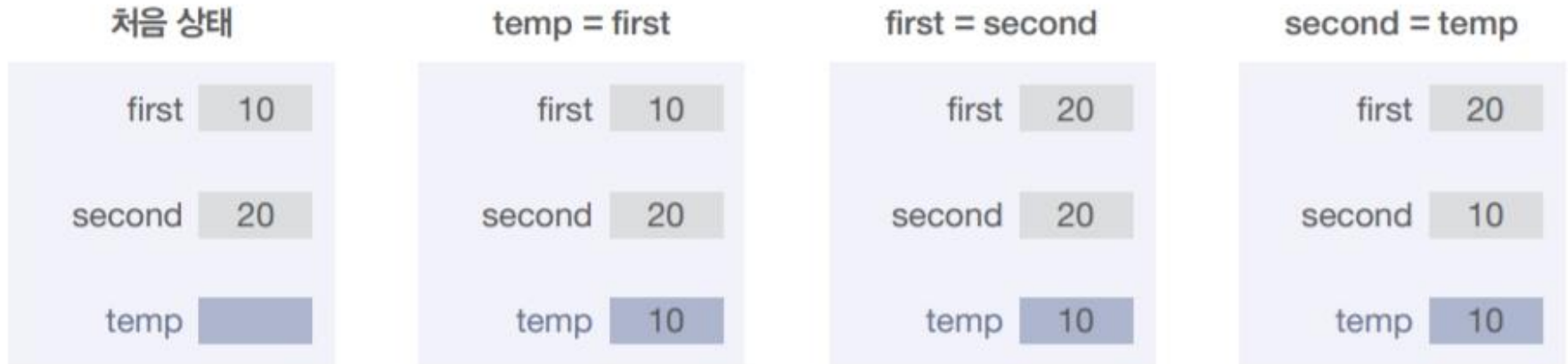


그림 6-17 추가적인 변수(temp)를 사용해서 두 변수 스왑하기

변수 스왑(swap)하기(1)

[Prg6-15.cpp 링크](#)

프로그램 6-15 변수 스왑(swap)하기

```
1  /*****
2  * This program shows how pass-by-reference can swap two values *
3  * in the calling function.                                     *
4  *****/
5  #include <iostream>
6  using namespace std;
7
8  void swap (int& first, int& second); // Function declaration
9
10 int main ( )
11 {
12     // Declaration
13     int first = 10;
14     int second = 20;
15     swap (first, second); // Function call
16     // Printing to test swapping
17     cout << "Value of first in main: " << first << endl;
18     cout << "Value of second in main: " << second;
19     return 0;
20 }
```

변수 스왑(swap)하기(2)

프로그램 6-15 변수 스왑(swap)하기

```
21  /*****
22  * The swap function takes two parameters as pass-by-reference *
23  * It then swaps the values of fst and snd using a temporary   *
24  * variable named temp. Since the data are passed by reference *
25  * changing the parameters in the swap means changing values in *
26  * the arguments (first and second in main)                      *
27  *****/
28  void swap (int& fst, int& snd)
29  {
30      int temp = fst;
31      fst = snd;
32      snd = temp;
33      return;
34  }
```

Run:

Value of first in main: 20

Value of second in main: 10

포인터로 전달(pass-by-pointer)(1)

- 포인터로 전달(pass-by-pointer) 메커니즘은 인수로 메모리 주소를 매개변수에 전달하는 것
- 주소를 전달하므로 매개변수를 사용해서 인수의 메모리 위치에 접근할 수 있음

OOP 하면 객체를 pointer 로 하면 겁나 힘들니까
웬만하면 pass - by - reference 해야함

장점과 단점

- a. Pass-by-value는 매우 간단하고, 호출되는 함수 쪽에서 인수를 조작하지 않게 차단

다만 인수의 값을 복사해서 매개변수로 전달하므로 전달해야 하는 값의 크기가 작다면 괜찮지만, 크다면 복사하는 작업이 무거워질 수 있음

따라서 객체 지향 프로그래밍에서는 객체의 크기가 클 때 값으로 전달 메커니즘을 사용하지 않음

포인터로 전달(pass-by-pointer)(2)

- b.** Pass-by-reference 전달 메커니즘은 호출되는 함수 쪽에서 매개변수를 변경해서, 호출한 함수 쪽의 원본(argument)을 변경할 수 있음. 스왑 등을 구현할 때 가장 좋은 선택지. 이 메커니즘은 복사가 필요하지 않다는 장점이 있음
pass-by-pointer : 메모리 확보함 / 4바이트만 확보하면 됨 (pointer 라서)
pass-by-reference : 메모리 확보할 필요 L L
- c.** Pass-by-pointer는 pass-by-reference 전달 메커니즘과 같은 장점을 가짐
- d.** 일반적으로 C++에서는 많이 사용하지 않지만, 전달해야 하는 자료가 포인터의 특성(예를 들어서 C 언어 문자열, 배열 등)을 갖고 있을 경우에는 포인터로 전달 메커니즘을 사용

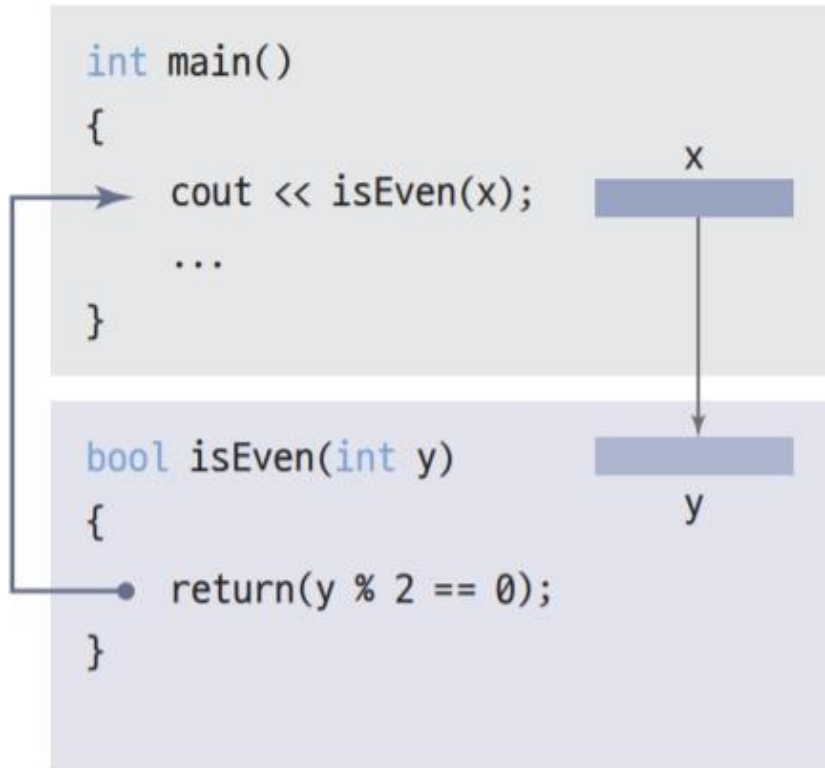
자료 리턴(1)

- 리턴값의 종류에 따라서도 값으로 리턴(return-by-value), 참조로 리턴(return-by-reference), 포인터로 리턴(return-by-pointer)이라는 3가지 메커니즘으로 구분

값으로 리턴(return-by-value)

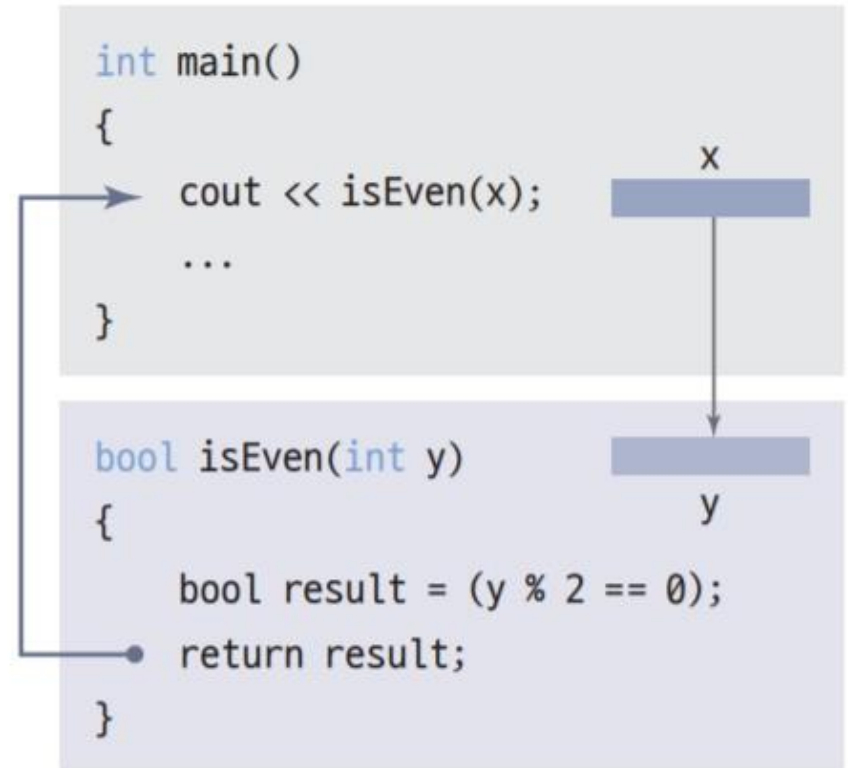
- 가장 일반적으로 사용되는 메커니즘은 값으로 리턴(return-by-value)
- 호출되는 함수 쪽에서 어떤 표현식을 생성하고, 이를 리턴
- 함수를 호출하면 값이 리턴되므로, 값이 필요한 위치에 함수를 활용

자료 리턴(2)



(a) 리터럴 값 리턴하기

그림 6-18 값으로 리턴(pass-by-value)의 개념



(b) 변수의 값 리턴하기

필기했음 관련 메모리 스택 참고 ㄱㄱ

리터럴 값 리턴하기(1)

[Prg6-16.cpp 링크](#)

프로그램 6-16 리터럴 값 리턴하기

```
1  /*****
2  * This program shows how we can return a value from a function *
3  *****/
4  #include <iostream>
5  using namespace std;
6
7  // Function declaration
8  bool isEven (int y);      책참고! 마들레이터 찾아서 보기
9
10 int main ( )              modulator : bool 형 자료형 변수는 원래 0/ 1 로만 출력되는데
11 {                          boolalpha 를 사용하면 true, false 로 출력한다.
12     // Function call
13     cout << boolalpha << isEven (5) << endl;
14     cout << boolalpha << isEven (10);
15     return 0;
16 }
17 /*****
18 * The isEven function takes one parameter y. It then checks to *
19 * see if the value of y is even by getting the remainder of y. *
20 * The function then returns the result by value (literal value) *
```

리터럴 값 리턴하기(2)

프로그램 6-16 리터럴 값 리턴하기

```
21  *****/
22  bool isEven (int y)
23  {
24      return ((y % 2) == 0);
25  }
```

Run:

false

true

참조로 리턴(return-by-reference) 클래스 객체 리턴시 사용

- 객체 지향 프로그래밍 메커니즘에서는 크기가 큰 객체를 리턴해야 하는 경우가 있음
- 이때 복사로 인해서 생하는 비용을 줄이려면, 참조로 리턴하는 것이 좋음
- 다만 호출되는 함수에서 객체를 생성하면, 함수가 종료된 이후에 객체가 사라지며 참조로 리턴을 할 때는 여러 주의 사항을 지켜야 함
- 포인터로 리턴(return-by-pointer)
 - Dynamic memory allocation시에 사용됨 (Ch09)
 - 동적메모리 할당을 을 하면 배열도 리턴할수 있다
dynamic memory allocation

실습

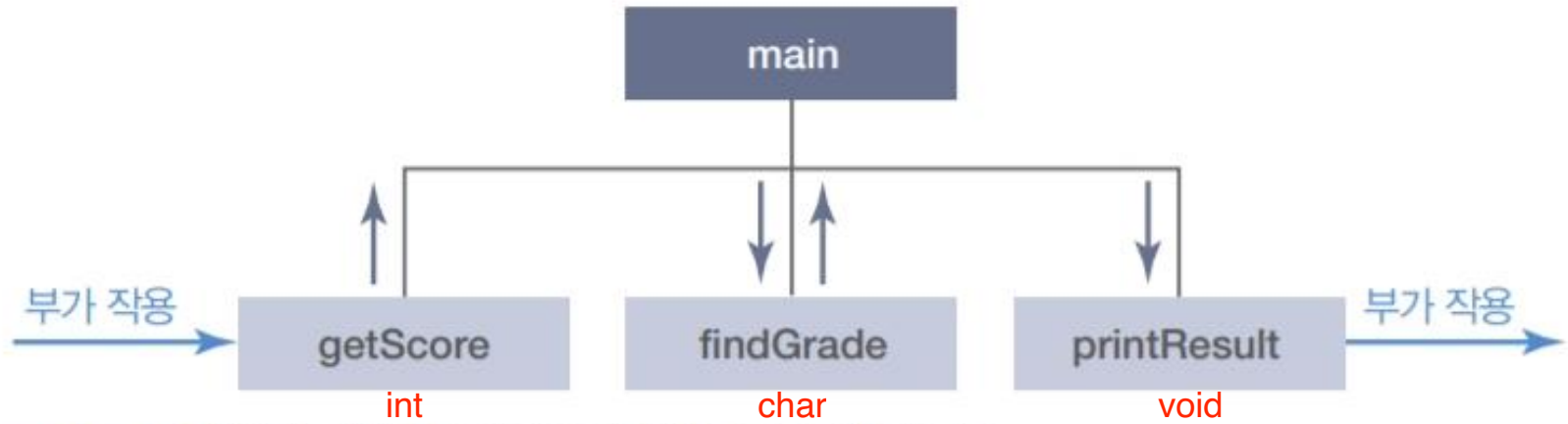


그림 6-19 점수를 기반으로 등급을 계산하는 프로그램의 구조

점수를 기반으로 등급 계산하기(1)

프로그램 6-17 점수를 기반으로 등급 계산하기

```
1  /*****
2   * This program shows how to use three functions to input,      *
3   * calculate, and print the score and grade of a student.      *
4   *****/
5  #include <iostream>
6  using namespace std;
7
8  // Function declarations
9  int getScore ();
10 char findGrade (int score);
11 void printResult (int score, char grade);
12
13 int main ()                file 이라는 커다란 scope 안에서 모든 function의 identify 가 달라야한다
14 {
15     // Declaration
16     int score;
17     char grade;
18     // Function calls
19     score = getScore ();
20     grade = findGrade (score);
```

점수를 기반으로 등급 계산하기(2)

프로그램 6-17 점수를 기반으로 등급 계산하기

```
21     printResult (score, grade);
22     return 0;
23 }
24 /*****
25  * The getScore function is an input function with a side effect *
26  * that gets the input from the user and returns the score by    *
27  * value to the main function. It uses a local variable (score) *
28  * whose value is returned to the main function. The function   *
29  * also validates the score to be between 0 and 100.             *
30  *****/
31 int getScore ()
32 {
33     int score; // Local declaration
34     do
35     {
36         cout << "Enter a score between 0 and 100: ";
37         cin >> score;
38     } while (score < 0 || score > 100);
39     return score;
40 }
```

점수를 기반으로 등급 계산하기(3)

프로그램 6-17 점수를 기반으로 등급 계산하기

```
41  /*****
42  * The findGrade function calculates and returns a character      *
43  * grade (A, B, C, D, F) related to the scored passed to it.    *
44  * It uses pass-by-value to get the value of the score. It      *
45  * use return-by-value to return the grade to the main function. *
46  * We have used a nested if-else construct, but it can also be  *
47  * be done using a switch statement.                             *
48  *****/
49  char findGrade (int score)
50  {
51      char grade; // Local declaration
52      if (score >= 90)
53      {
54          grade = 'A' ;
55      }
56      else if (score >= 80)
57      {
58          grade = 'B' ;
59      }
60      else if (score >= 70)
```

점수를 기반으로 등급 계산하기(4)

프로그램 6-17 점수를 기반으로 등급 계산하기

```
61     {
62         grade = 'C' ;
63     }
64     else if (score >= 60)
65     {
66         grade = 'D' ;
67     }
68     else
69     {
70         grade = 'F' ;
71     }
72     return grade;
73 }
74 /*****
75  * The last function prints the score and corresponding grade. *
76  *****/
77 void printResult (int score, char grade)
78 {
79     cout << endl << "Result of the test." << endl;
80     cout << "Score: " << score << " out of 100"<< endl;
```

점수를 기반으로 등급 계산하기(5)

프로그램 6-17 점수를 기반으로 등급 계산하기

```
81     cout << "Grade: ";  
82 }
```

Run:

Enter a score between 0 and 100: 87

Result of the test.

Score: 87 out of 100

Grade: B

Run:

Enter a score between 0 and 100: 93

Result of the test.

Score: 93 out of 100

Grade: A

함수 오버로딩(1)

- 이름이 같은 함수를 2개 정의할 수 있을까요?
- 매개변수(매개변수의 자료형, 개수, 순서)가 다르다면 가능
- 이를 함수 오버로딩(function overloading)이라고 부름
- 프로그램이 같은 이름의 함수를 허용할 때, 함수들을 구분하기 위해 사용하는 기준을 함수 시그니처(function signature)라고 부름

함수의 Signature

- 함수를 특징 지우는 요소
 - 함수의 이름
 - 함수의 매개변수의 개수, 타입
 - **Return Type, pass by reference**는 signature에 포함 않됨
 - 서로 다른 함수임을 특징지우는 요소
-
- `Max_d(doubl a, double b);`
 - `Max_i(int a, int b);`
 - `Max_c(char a, char b);`
-
- `Max(doubl a, double b);`
 - `Max(int a, int b);`
 - `Max(char a, char b);`
- 이름은 같지만 시그니처는 다르기 때문에 완전히 다른 함수이다
같은 scope 내에 선언될수 있다

오버로딩(Overloading)

- 여러함수들이 동일한 이름을 사용할 수 있는 기능
- 함수이름이 같아도 Signature가 다르다면 이 함수들은 서로 오버로드되어 사용할 수 있는 서로 다른 함수임.

오버로딩(Overloading)

- 오버로딩을 사용해서 두 가지 버전의 max() 함수를 만드는 예

```
void max( int a, int b );  
float max( float a, float b );
```

- 오버로딩을 사용할 수 없다면 서로 다른 이름의 함수를 만들어서 사용해야 한다.

```
void max_i( int a, int b );  
float max_f( float a, float b );
```

기본적인 오버로딩의 규칙

- 인자의 타입을 통해서 호출될 함수를 결정한다.

```
int VariableReturnType( char c, int i );    // 1번
double VariableReturnType( char c, double d );    // 2번
...
VariableReturnType( 'A', 100 );    // 1번 함수 호출
VariableReturnType( 'A', 100.12 );    // 2번 함수 호출
```

반환값은 함수의 시그니처가 될수 없기 때문이다

- 반환 값**만 틀린 경우는 오버로드 할 수 없다.

```
int VariableReturnType( char c, int i );
double VariableReturnType( char c, int i ); // Error
...
VariableReturnType( 'A', 100 );    // 어느 함수를 호출해야 할 지 모름
```

기억하기!!

- Pass-by-ref인 경우도 오버로드 할 수 없다

```
void SameSignature( int i );
void SameSignature( int& r );
...
int i = 10;
SameSignature( i );    // 어느 함수를 호출해야 할 지 모름
```

함수 오버로딩(2)

- 함수 시그니처는 매개변수들의 자료형과 조합
- 이름이 같은 함수라도 함수 시그니처가 다르다면, 호출 시점에 컴파일러가 자신이 어떤 함수를 호출해야 하는지 구분할 수 있음

함수 오버로딩(3)

- 다음은 두 int 자료형과 double 자료형의 최대값을 찾기 위해 다른 함수를 정의하는 예시

```
int max(int a, int b)
{
    ...
}
```

```
double max(double a, double b)
{
    ...
}
```

- 다음 두 함수는 함수 시그니처가 같으므로 오버로딩할 수 없음

```
int get()
{
    ...
}
```

```
double get()
{
    ...
}
```

함수 3개 오버로딩하기(1)

프로그램 6-19 함수 3개 오버로딩하기

```
1  /*****
2  * This program shows how we can use different signatures to      *
3  * create three overloaded functions.                               *
4  *****/
5  #include <iostream>
6  using namespace std;
7
8  // Function Declaration
9  int max (int num1, int num2);
10 int max (int num1, int num2, int num3);
11 int max (int num1, int num2, int num3, int num4);
12
13 int main ()
14 {
15     // Three calls to overloaded max functions
16     cout << "maximum (5, 7): " << max (5, 7) << endl;
17     cout << "maximum (7, 9, 8): " << max (7, 9, 8) << endl;
18     cout << "maximum (14, 3, 12, 11): " << max (14, 3, 12, 11);
19     return 0;
20 } // End main
```

함수 3개 오버로딩하기(2)

프로그램 6-19 함수 3개 오버로딩하기

```
21  /*****
22  * The following is the definition for the max function with *
23  * two arguments num1 and num2. It returns the maximum of the *
24  * parameters. *
25  *****/
26  int max (int num1, int num2)
27  {
28      int larger; // Local variable
29      if (num1 >= num2)
30      {
31          larger = num1;
32      }
33      else
34      {
35          larger = num2;
36      }
37      return larger;
38  }
39  /*****
40  * The following is the definition for the max function with *
```

함수 3개 오버로딩하기(3)

프로그램 6-19 함수 3개 오버로딩하기

```
41  * three arguments num1, num2, and num3. It uses a call to      *
42  * the function with two arguments to first find the maximum    *
43  * of the first two. It then makes another call to the first    *
44  * function to find the maximum of the result and the num3.     *
45  *****/
46  int max (int num1, int num2, int num3)
47  {
48      return max(max(num1, num2), num3);
49  }
50  /*
51  * The following is the definition for the max function with    *
52  * four arguments num1, num2, and num3, and num4. It uses a      *
53  * a call to the function with three arguments first to find     *
54  * the maximum of the first three, and then call the function    *
55  * with two arguments to find the max of the previous result     *
56  * and the fourth number.                                         *
57  *****/
58  int max (int num1, int num2, int num3, int num4)
59  {
60      return max(max(num1, num2, num3), num4);
```

함수 3개 오버로딩하기(4)

프로그램 6-19 함수 3개 오버로딩하기

```
61 } // End max function with four parameters
```

Run:

```
maximum (5, 7): 7
```

```
maximum (7, 9, 8): 9
```

```
maximum (14, 3, 12, 11): 14
```